

Lab 3: Serial Communication, Non-Preemptive Scheduling, and ISRs

Basic information

Names and students ID

Nathan Joslin, njoslin

Xiqiang Huang, xiqiang

Turn-in date: 05/08/2026

Assignment: Interrupts, Scheduling, and BLE Communication Using ESP32

1. Introduction

In this lab we will explore several important concepts, including I2C serial communication, non-preemptive task scheduling, and interrupt service routines (ISRs) using the ESP32-S3 microcontroller.

There are 3 parts in this lab. The first part is to implement the I2C communication between the esp32 and the LCD. Instead of using many pins to control the LCD, this lab used an I2C LCD module containing a PCF8574I/O expander, which can let us control the behavior of the LCD using only two lines: SDA (Serial Data) and SCL (Serial Clock). The key of part 1 is to handle 4-bit transmitted communication through the LCD and bitwise operation during this process.

Part 2 focuses on implementation of a basic non-preemptive priority scheduler. Non-preemptive priority scheduling is an algorithm where a process, once allocated the CPU, holds it until it terminates or voluntarily waits. No interrupt happened during this process. There are 5 tasks in this lab, which are Task A, Task B, Task C, Task D and Task E. Different tasks have their own behaviors and priorities. The scheduler needs to manage 4 tasks based on the priority, and if they have the same priority, it will use the round robin algorithm.

The last part of the lab introduces 3 different interrupts when the CPU deals with different tasks. Three interrupts are timer, push-button, BLE communication callback interrupt. The objective of this section is to learn how to develop and manage interrupts and ISRs for task handling.

2. Methods and Techniques

Part I, We need to display a message to the LCD device entered through the Serial Monitor using I2C communication. The LCD instructions operate in an 8-bit format, but the communication between the LCD and the ESP32 occurs in 4-bit mode. In other words, using this way of communication, every character from the serial monitor had to be split into two 4-bit nibbles. Our approach is to send the upper 4 bits sent first, and then send the lower 4 bits. Finally, we combine two parts together by doing bit operations.

In Part II, we need to implement a non-preemptive priority scheduling using task control blocks. For this part, we do it step by step in the following orders

- Building the TCB block and state block. The TCB block contains 5 fields, including function pointer, name of the task, priority, task state, and wake time (when the task will run again). For the state block, it contains four states, which are RUNNING, READY, HALTED, SLEEPING.
- Defining the sleep and halt_me function. Sleep function will take sleeping time duration as an argument. It will set the state of the task to SLEEPING, and wake it up after the millis() + sleeping time duration. And the halt_me function will set the state to HALTED, and it will mark the task finished.

- Defining 5 different tasks based on the given instructions, including taskA, taskB ,taskC, taskD and TaskE.
- Creating a scheduler. This is the most difficult part of this section. The main idea is to find the max priority task every time, and execute it, if every task has a different priority. If they share the same priority, the scheduler will continuously check the task states and select the next task to execute based on both priority level and task availability.

For Part III, the basic architecture of the code is shown below,

Interrupt happens	->	ISR/cal l back runs	->	Set flag	->	loop()	->	Checks flag	->	LCD updates
-------------------	----	---------------------	----	----------	----	--------	----	-------------	----	-------------

The trick part of this section is to pause the counter for 2 seconds when the interrupt happens. Our approach is to build a pause function that temporarily stops the counter update while displaying the interrupt message on the LCD. After the 2-second delay expired, the counter resumed normal operation.

Experimental Results

Part I, the LCD successfully displays the content based on our input in the Serial monitor.

Taking the following figures for example. We enter “Hello,ECE 474” Figure 1, showing us there are 14 characters(including the space at the end)in total. And Figure 2 shows the exact content as our input.

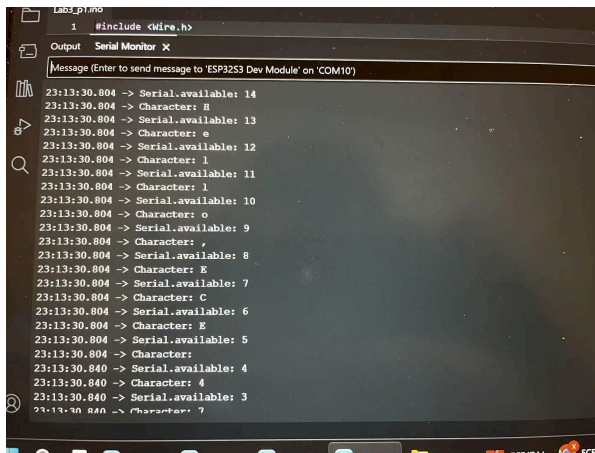


Figure 1(left), showing how the serial read the data, and Figure 2(right) showing the content from the Serial monitor

For part 2, 5 different tasks have various behaviors and running time.

From Figure 3, we can see the taskB and TackC are executed . For TaskB , the count can only go up to 10, and for taskC, using the lcd to display the playing note of the melody.

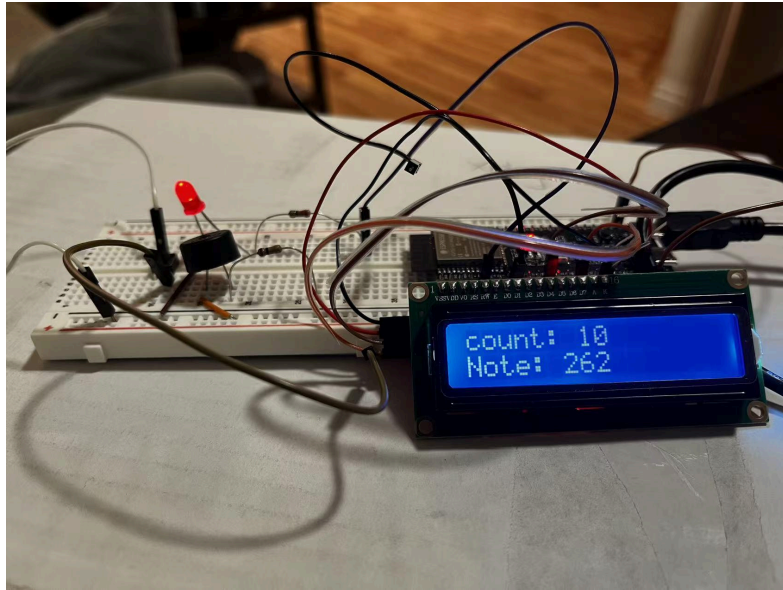


Figure 3. The LCD showing the count and the note is being played

Figure 4 is showing how the program executes the tasks in one round. We can see the Alphabet printer is doing 2 rounds . and the music play is executed one time. As the LED is running forever, we do not see the taskA is finished. On the other hand, the message of the task C(lcd counter) finished did not appear as frequently as the other tasks. This occurred might be Task B used a longer sleep interval of 2 seconds between executions, causing it to progress much more slowly than the Music Player and Alphabet Printer tasks

```

23:37:33.633 -> A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, Finished: MUSIC PLAYER, priority: 2
23:37:40.982 -> P, Q, R, S, T, U, V, W, X, Y, Z,
23:37:46.475 -> A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z,
23:37:58.996 -> Finished: Alphabet Printer, priority: 3
23:38:03.448 -> New Priority Scheme: 2
23:38:03.492 -> A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, Finished: MUSIC PLAYER, priority: 2
23:38:10.978 -> P, Q, R, S, T, U, V, W, X, Y, Z,
23:38:16.460 -> A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z,
23:38:28.990 -> Finished: Alphabet Printer, priority: 3
23:38:33.445 -> New Priority Scheme: 3
23:38:33.480 -> A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, Finished: MUSIC PLAYER, priority: 2
23:38:41.003 -> P, Q, R, S, T, U, V, W, X, Y, Z,
23:38:46.480 -> A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z,
23:38:59.273 -> Finished: Alphabet Printer, priority: 2
23:39:03.444 -> New Priority Scheme: 1
23:39:03.483 -> A, B, C, D, E, F, G,

```

Figure 4. The process of the tasks being executed

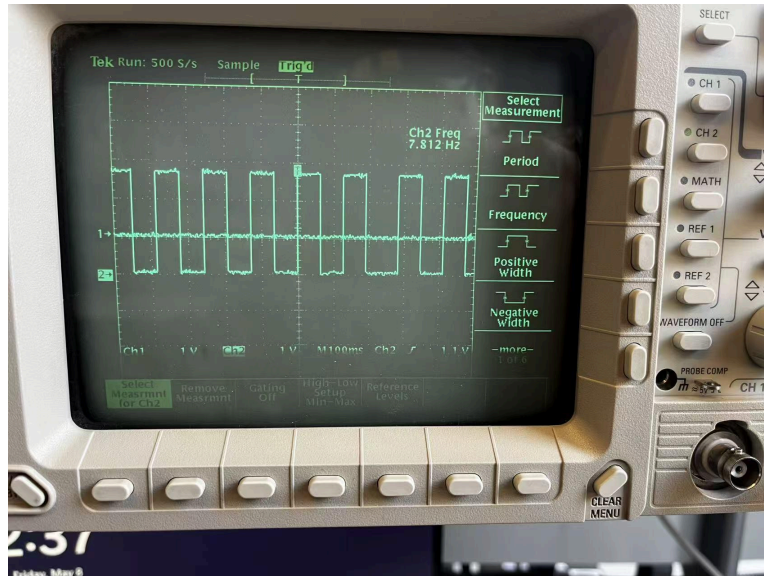


Figure 5. The waveform of the voltage for the LED in part 2

From Figure 5, the frequency of the LED is around 7.8Hz, which is close to the 8 Hz required in the lab.

For TaskA, TaskB, and TaskC, we have a 30s- video for the demos:lab3_p2

Source:<https://youtu.be/uRuSDINRnTw?si=rHqOMVX1rxITd9x3>

For Part III of the lab,

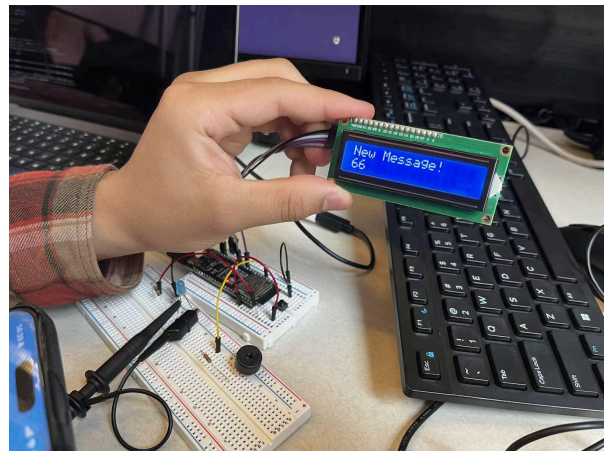


Figure 6,(left), showing when the button is being pressed, and Figure 7(right), showing when we send the message to the esp32.

More demos for this section of the lab is in this 30s- video:

<https://youtu.be/23S7cXFhUGg?si=Jj2TpEQaVWSh5hjI>

D. Code Documentation

Document the major software components:

- LCD initialization
- Button ISR
- Timer ISR
- BLE callback
- pause_ms() function
- Main loop flag handling
- Mention important functions and line ranges

E. Overall Performance Summary

Discuss successes, failures, debugging steps, and how well the system met the lab objectives.

Overall, our systems meet the requirements for this lab, but this came with some struggle. Parts 1 and 3 were fairly straightforward, but Part 2 proved to compound in difficulty quickly. In this part, we built small components, tested them, and slowly added them into the system piece by piece. The first struggle was getting a basic scheduler working. To do so, we first needed to ensure we understood what a scheduler actually does and not overload it with too many responsibilities. It simply needed to look at signals and command tasks to run based on those signals. To test its functionality, we wrote dummy tasks that printed simple text to the console and observed which tasks fired off and when. Once this was tested and implemented, we moved on to the Round-Robin scheduler.

The Round-Robin algorithm was a bit niche because it is intended for preemptive scheduling, not non-preemptive scheduling. Therefore, a lot of the core pieces that make Round-Robin work were stripped down to comply with Part 2's problem while still accounting for when tasks have equivalent priorities. At first, we tried simply detecting if the previous task and the current task had the same priority. If so, we would give priority to the current task, but this algorithm proved to be very convoluted. One thing we observed was that we did not necessarily need to check for equivalent priorities, but instead change where the scheduler begins looking for the next task. To elaborate, if two tasks had the same priority, we would simply move the offset to the next task. This avoids starvation. To ensure a successful implementation, we started by adding two tasks into the scheduler, testing to ensure they were getting optimal runtime, and then adding more tasks one at a time. To conclude, Part 2 was very difficult, but it helped us learn a lot.

F. Teamwork Breakdown

Describe each team member's responsibilities including coding, hardware setup, debugging, and testing.

As a team, we each did our own implementations so that we both had a full understanding of the systems and were on the same page.

Team Roles

Activity	Student(s) Name
Coding	Nathan, Xiqiang
Hardware Setup	Nathan, Xiqiang
Debugging	Nathan, Xiqiang
Testing	Nathan, Xiqiang

Discussion and Conclusions

The most challenging part of this lab is to implement the non-preemptive scheduler part II.

It needs to manage multiple tasks with different or the same priority. Compared to the equal priority, the different priorities are more challenging because the scheduler had to continuously determine which READY task had the highest priority and ensure that lower-priority tasks were still able to execute. For example in scheme 3, where all the tasks have same priority, implementation using round-robin scheduling is pretty hard for me.

Another challenge is to correctly restart halted tasks after the scheduler switched to a new priority scheme. For this part, I use the matrix to store the priority of the scheme, and use the round-robin algorithm to select the next task we need to execute.

One part that we are most proud of is that we successfully implement all tasks we need in this lab, including the LCD display, LED blinking, buzzer producing different melodies, and using 3 different interrupts. Even most of the time we do not get the expected output, but we just keep trying. During this process, all types of issues showed up. For me, even the connection between the board and the arduino happens very often. I have to search for some information to solve this problem, just lucky I did not break the board. For my teammate Nathan, he did a lot of debugging for part 2 of the lab. He tries to print out the order of the tasks being executed by the processor. But in one afternoon, he always gets the wrong output. What he did just to read over the material again and search for more information online, and keep trying.

Overall, this lab taught us the importance of patience, debugging, and persistence we should have during the development process in the embedded system.